

Time left : 00:58

Question 2

What is the output of the following C code:

Code

```
#include <stdio.h>

void fun(int(*)());
int main()
{
    fun(main);
    printf("Earth\n");
    return 0;
}
void fun(int(*p)())
{
    printf("Hacker");}
```

☐ HackerEarth

☐ Earth

Time left : 00:26

Question 3

In an operating system, which of these options refer to the area in memory that is used as a pool for dynamic memory allocation?

☒ Stack

☐ Heap

☐ Virtual memory

☐ Data segment

[Reset Answer](#)

[Previous Question](#)

Time left : 00:55

Question 4

Max. sco

A bucket has 4 apples, 3 oranges, 5 bananas, and 2 mangoes. A person draws 4 fruits at random. What is the probability that out of four fruits, at least 1 is orange?

☐ 67/1001

☐ 330/1001

☐ 288/343

☐ 55/343

[Reset Answer](#)

[Previous Question](#)

Time left : 00:23

Question 5

Bob is working on a project that involves networking. He wants to test the IP stack on his localhost, which IP address should he ping?

☐ 127.0.0.0

☐ 1.0.0.127

☐ 127.0.0.1

☐ 127.0.0.255

[Reset Answer](#)

[Previous Question](#)

Explanation

Given

- $N = 2$

- $Q = 5$

- Operations = $\{(1, 2, -1), (2, 1, 3), (2, 2, 4), (2, 4, 5), (1, 2, 4)\}$

Approach

- During the 1st operation of type 1: The cache is empty. Hence, for query $(1, 2, -1)$, you get the value -1.
- During the 2nd operation of type 2: You have $1 \rightarrow 3$ in the cache.
- During the 3rd operation of type 2: You have $1 \rightarrow 3$ and $2 \rightarrow 4$ in the cache.
- During the 4th operation of type 2: You have $2 \rightarrow 4$ and $4 \rightarrow 5$ in the cache. Since the frequency of key 1 and key 2 was the same, key 1 was removed.
- During the 5th operation of type 1: You get a value of 4.

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 1.0 sec(s) for each input file

Memory Limit: 256 MB

Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

Allowed Languages

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml, Octave, Pascal, Perl, PHP, Python, Python 3, Python 3.8, Racket, Ruby, Rust, Scala, Swift, TypeScript, Visual Basic

[Previous Question](#)

[Next Question](#)



Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains N denoting the capacity of the cache.
- The next line contains Q denoting the number of operations on the cache.
- The next Q lines contain 3 space-separated integers.

Output format

For each query of type 1, print the space-separated values on a single line.

Constraints

$$1 \leq N \leq 5 \times 10^4$$

$$1 \leq Q \leq 2 \times 10^5$$

$$1 \leq K \leq 2 \times 10^5$$

$$1 \leq \text{value} \leq 10^9$$

Sample input

```
2
5
1 2 -1
2 1 3
2 2 4
2 4 5
1 2 -1
```

Sample output

```
-1 4
```

Explanation

Given

[Previous Question](#)

[Next Question](#)

Cache it

A cache is a limited storage that stores key-value pairs. It uses a eviction policy to manage its size. Least Frequently Used (LFU) cache removes the least frequently accessed key when the capacity is reached.

You need to implement the Least Frequently Used (LFU) cache with the capacity N .

You are also given Q operations of the following type:

- 1. **key, val**: Get the value of the key from the cache. If the value does not exist in the cache, return -1.
- 2. **key, value**: Update the value of the key if present, or insert the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie (two or more keys with the same frequency), the smallest key should be removed.

For each operation of type 1, print the required value.

Function description

Complete the function `solve()`. This function takes the following 3 parameters and returns the required array of results:

- N : Represents the capacity of the cache.
- Q : Represents the number of operations on the cache.
- `operations`: Represents the operations on the cache.

Input format for custom tester:

[Previous Question](#)

[Next Question](#)

C++17 (g++ 10.3.0)

Auto-complete ready!

```
1 > #include<bits/stdc++.h> ...  
3 int solve (int N, vector<char> colors) {  
4     return 1;  
5 > }
```

Explanation

- Initially, Points = 0
- In operation 1, remove crayon set of color b from index 2 to 4, colors = [a, a, a, a, c] Points = $0 + 2 \times 2 = 4$.
- In operation 2, remove crayon set of color c at index 4, colors = [a, a, a, a] Points = $4 + 1 \times 1 = 5$.
- In operation 3, remove crayon set of color a from index 0 to 3, colors = [] Points = $5 + 4 \times 4 = 21$.
- Hence, the maximum point is 21.

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 5.0 sec(s) for each input file

Memory Limit: 256 MB

Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes

Allowed Languages

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml, Octave, Pascal, Perl, PHP, Python, Python 3, Python 3.8, Racket, Ruby, Rust, Scala, Swift, TypeScript, Visual Basic

[< Previous Question](#)

[Next Question >](#)



- The first line contains an integer N denoting the total number of crayons in the box.
- The second line contains N space-separated characters denoting the color of the crayons in the box.

Output format

Return the maximum number of points you can get in the given scenario.

Constraints

$$1 \leq N \leq 60$$

colors contain only lowercase English letters.

Sample input

```
7
a a b b a a c
```

Sample output

```
21
```

Explanation

- Initially, Points = 0
- In operation 1, remove crayon set of color b from index 2 to 4. colors = [a, a, a, a, c]. Points = $0 + 2 \times 2 = 4$.
- In operation 2, remove crayon set of color c at index 4. colors = [a, a, a, a]. Points = $4 + 1 \times 1 = 5$.
- In operation 3, remove crayon set of color a from index 0 to 3. colors = []. Points = $5 + 4 \times 4 = 21$.
- Hence, the maximum point is 21.

Crayon Removal Scoring Game

You have a container filled with crayons of various colors, where each color is represented by a different alphabet letter. In each move, you can eliminate a sequence of adjacent crayons that share the same color. You may repeat this process multiple times until the container becomes empty. For each move, you must select a continuous group of identical colored crayons (group of x crayons, where $x \geq 1$). Upon removing them, you earn $x \cdot x$ points.

You are provided with an integer **N** representing the total count of crayons in the container. You also receive an array **colors** containing the **N** colors in the container, where each color is denoted by an English alphabet letter.

Calculate the maximum points you can achieve in this scenario.

Function description

Complete the **solve()** function. This function takes the following 2 arguments and returns the answer.

Parameters:

- **N**: Represents the total crayons in the box
- **colors[]**: Represents the array of colors of crayons in the box. Each color is represented by an English alphabet.

Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains an integer **N** denoting the total number

Explanation

We are given $N = 5$ and $K = 2$. The array is: $[1, 2, 1, 2, 1]$. Now, we need to count all subarrays where at least one number appears 2 times or more.

Valid dominant subarrays:

1. $[1, 2, 1] \rightarrow$ (1 appears twice)
2. $[2, 1, 2] \rightarrow$ (2 appears twice)
3. $[1, 2, 1] \rightarrow$ (1 appears twice)
4. $[1, 2, 1, 2] \rightarrow$ (1 and 2 both appear twice)
5. $[2, 1, 2, 1] \rightarrow$ (1 and 2 both appear twice)
6. $[2, 1, 2, 1, 2] \rightarrow$ (2 appears 3 times)

Thus, the correct output is 6.

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 5.0 sec(s) for each input file

Memory Limit: 256 MB

Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passed.

Allowed Languages

Bash, C, C++14, C++17, Clojure, C#, D, Erlang, F#, Go, Groovy, Haskell, Java 8, Java 14, JavaScript(Node.js), Julia, Kotlin, Lisp (SBCL), Lua, Objective-C, OCaml, Octave, Pascal, Perl, PHP, Python, Python 3, Python 3.8, Racket, Ruby, Rust, Scala, Swift, TypeScript, Visual Basic.

[Previous Question](#)

[Next Question >](#)

DXGA

165Hz 3ms

NVIDIA

G-SYNC

Sample input

Sample output

```
5
2
1 2 1 2 1
```

6

Explanation

We are given $N = 5$ and $K = 2$. The array is $[1, 2, 1, 2, 1]$. Now, we need to count all subarrays where at least one number appears 2 times or more.

Valid dominant subarrays:

1. $[1, 2, 1] \rightarrow$ (1 appears twice)
2. $[2, 1, 2] \rightarrow$ (2 appears twice)
3. $[1, 2, 1] \rightarrow$ (1 appears twice)
4. $[1, 2, 1, 2] \rightarrow$ (1 and 2 both appear twice)
5. $[2, 1, 2, 1] \rightarrow$ (1 and 2 both appear twice)
6. $[2, 1, 2, 1, 2] \rightarrow$ (2 appears 3 times)

Thus, the correct output is 6.

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 5.0 sec(s) for each input file
Memory Limit: 256 MB
Source Limit: 1024 KB

Scoring

Score is assigned if any testcase passes.

[Previous Question](#)

[Next Question >](#)

Function Description

You must implement the function `countDominantSubarrays`.

? Parameters

`int N` - The number of elements in the array. `int K` - The minimum required frequency for a number to make a subarray dominant. `int[] arr` - An array of size `N`, containing the elements.

? Returns

`int` - The total number of **dominant subarrays** in the given array.

Input Format

1. **First Line:** An integer **N** (size of the array).
2. **Second Line:** An integer **K** (the minimum frequency required for a subarray to be dominant).
3. **Third Line:** **N** space-separated integers (the array elements).

Output Format

Print a single integer, representing the **number of dominant subarrays**.

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq K \leq N$
- $1 \leq A[i] \leq 10^9$
- **Time Limit:** 2 seconds
- **Memory Limit:** 256 MB

Sample input

Sample output



Question 6

Max. score: 40.00

Dominant Subarrays

You are given an array of **N** integers. A subarray is called **dominant** if at least one number appears **K times or more** within it.

Your task is to count the number of such **dominant subarrays** in the given array.

- You have an **array of size N**.
- A **subarray** is a **continuous** part of the array (it cannot skip elements).
- A **dominant subarray** is one where at least **one number appears K times or more**.
- Your goal is to **count how many such dominant subarrays exist**.

Function Description

You must implement the function `countDominantSubarrays`.

? Parameters

int N - The number of elements in the array. int K - The minimum required frequency for a number to make a subarray dominant. int[] arr - An array of size N, containing the elements.

? Returns

int - The total number of **dominant subarrays** in the given array.

Input Format

1. **First Line:** An integer **N** (size of the array).